

April 2025

Compilers Project

Team 17

Team Info

Name	Code	Sec	BN
Mohamed Yasser Mohamed	9211066	2	12
Mustafa Tarek Salah	9211178	2	22
Daniel Nabil Khalil	9210386	1	16
Ahmed Bassem Wassim	9210048	1	7

Project Overview

The **Mini C Compiler** is a simplified programming language compiler inspired by C, designed with a minimal set of modifications.

- **Supported Data Types:**

- Integers
- Floats
- Booleans
- Characters
- Strings

- **Core Language:**

- Variable and Constant Declarations
- Expressions:
- Assignment Statements:
- Control Flow:
 - if-else statements
 - while loops
 - do-while loops
 - for loops
 - switch statements
- Block Structure
- Functions

How to run our compiler

1. Running from the Terminal:

- Make sure you have Flex, Bison, and the GCC compiler installed on your Ubuntu system.
- Open a terminal, navigate to the project directory, and simply run:
 - `./build.sh`
- This will build and execute the compiler through the terminal.

2. Running with the GUI:

- Ensure Python and Tkinter are installed.
- In the terminal, run:
 - `python gui.py`
- This will launch the application window.
- Paste or write your code in the editor and click Compile to process your code through the compiler.

You can see the compiler outputs (errors, warnings, tokens, symbol_table, and quadruples) in "output" folder.

Tools and Technologies

- Compiler : GCC
- Lexer: Flex
 - Flex is a fast and efficient scanner generator that is used to automatically generate the lexical analyzer.
- Parser: Bison
 - Bison is a parser generator that works similarly to Yacc. It is used to generate the syntax analyzer for the Mini C Compiler, enabling the interpretation of grammar rules and the parsing of source code into meaningful constructs.
- GUI: Tkinter
- Tkinter is a standard Python library for creating graphical user interfaces.

Tokens

Token	Description
INT	The type keyword 'int'
FLOAT	The type keyword 'float'
STRING	The type keyword 'string'
CHAR	The type keyword 'char'
BOOL	The type keyword 'bool'
VOID	The type keyword 'void'
CONSTANT	The keyword 'const'
INTEGER_VALUE	Integer literal, e.g. '42'
FLOAT_VALUE	Float literal, e.g. '3.14'
BOOL_VALUE	Boolean literal, 'true' or 'false'
STRING_VALUE	String literal, e.g. "hello"

Tokens

Token	Description
CHAR_VALUE	Character literal, e.g. 'A'
ADD	+
SUB	-
MUL	*
DIV	/
MOD	%
POW	^
ASSIGN	=
EQUAL	==
NOT_EQUAL	!=
GREATER_THAN	>

Tokens

Token	Description
LESS_THAN	<
GREATER_EQUAL	>=
LESS_EQUAL	<=
LOGICAL_AND	&&
LOGICAL_OR	
LOGICAL_NOT	!
BITWISE_AND	&
BITWISE_OR	
BITWISE_NOT	~
SHIFT_LEFT	<<
SHIFT_RIGHT	>>

Tokens

Token	Description
PRE_POST_INC	++
PRE_POST_DEC	--
SEMICOLON	;
COMMA	,
IF , ELSE	if , else
FOR , WHILE , DO_WHILE	for , while , do while
SWITCH , CASE , DEFAULT	switch , case , default
BREAK , CONTINUE , RETURN	break , continue , return
IDENTIFIER	variable/function names
UNKNOWN_TOKEN	Anything Else

Tokens

Token	Description
ADD_ASSIGN	+=
SUB_ASSIGN	-=
MUL_ASSIGN	*=
DIV_ASSIGN	/=
MOD_ASSIGN	%=
POW_ASSIGN	^=
LOGICAL_AND_ASSIGN	&&=
LOGICAL_OR_ASSIGN	=
BITWISE_AND_ASSIGN	&=
BITWISE_OR_ASSIGN	=
BITWISE_NOT_ASSIGN	~=
SHIFT_LEFT_ASSIGN SHIFT_RIGHT_ASSIGN	<<= >>=

Quadruples

Quadruple	Description
push 'value'	Push a value (constant or variable) onto the stack
pop 'destination'	Pop the value from stack and assign to destination
add	Pop two values from the stack, push their sum
sub	Pop two values from the stack, push their subtraction
mul	Pop two values from the stack, push their multiplication
div	Pop two values from the stack, push their division
mod	Pop two values from the stack, push their remainder
pow	Pop two values from the stack, push their exponentiation result
neg	Negate the value on top of the stack
logical_and	Logical AND (&&) of two boolean values on the stack
logical_or	Logical OR () of two boolean values on the stack

Quadruples

Quadruple	Description
logical_not	Logical NOT (!) of the value on the stack
bitwise_and	Bitwise AND (&) of two integer values on the stack
bitwise_or	Bitwise OR () of two integer values on the stack
bitwise_not	Bitwise NOT (~) of the value on the stack
shift_left	Bitwise left shift of an integer value
shift_right	Bitwise right shift of an integer value
equal , not_equal	Comparison operations (==, !=) between two values on top of the stack, pushing boolean result to stack
greater_than, greater_equal, less_than, less_equal	Relational comparisons, push boolean result
post_inc, post_dec	Post-increment/post-decrement operation on variable
pre_inc, pre_dec	Pre-increment/pre-decrement operation on variable
print	Print the value currently on top of the stack or in argument

Quadruples

Quadruple	Description
call func	Call function 'func'
end func	Mark end of function 'func'
return	Return from the current function
continue	jump to the start of the current loop
jf <label>	"Jump if false": If top of stack is false, jump to <label>
jmp <label>	Unconditional jump to label
start_<type>_<n>:	Marks the start of a loop or block.
end_loop_<n>:	Marks the end of a loop
end_if_<n>	Marks the end of an if-statement's true block.
false_label_<n>:	Label target for failed conditions in if, while, etc.

Semantic Checks

- **Undeclared Identifier Check**
- **Identifier Redclaration Check**
- **Constants Must Be Initialized**
- **Assignment to Constants Not Allowed**
- **Variable Must Be Initialized before Use**
- **Type Compatibility for Assignments**
 - `int ↔ float` (allowed)
 - `int ↔ bool` (allowed)
 - `float ↔ bool` (allowed)
 - `char → string` (allowed)
 - `string → char` (not allowed)
 - `void ↔ non void` (disallowed)
- **Type Compatibility for Expressions**
 - `int ↔ float` (allowed)
 - `int ↔ bool` (allowed)
 - `float ↔ bool` (allowed)
 - `char ↔ string` (allowed) for concatenation
- **Function Argument Count Check**
 - Function calls must have exactly the correct number of arguments.
- **Function Argument Type Check**
- **Function Return Type Checks**
 - A non-void function must contain at least one return statement with a value of the correct type.
 - A void function cannot return a value.
- **Unused Symbol Warnings**
- **Scope Handling**
 - When a block ends, variables declared in that block go out of scope (no explicit error now, but future use will cause undeclared identifier error).
- **Disallowed Operations**
 - Arithmetic on string or char or void
 - Boolean logic on string, char, void
 - Bitwise on types other than int
- **All semantic errors are reported with line numbers and clear messages, both to console and to an error file.**